KnowOps AI: Phase 2.2 Architecture Report

Component: Knowledge Base Ingestion Pipeline (scripts/ingest.py) & IT Operations Runbooks
Date: October 2026 | **Sprint:** sprint-1-(base) / Phase 2

---

1. Executive Summary & Purpose

Phase 2.2 built the **Knowledge Base Ingestion Pipeline** — the mechanism that converts raw IT operations documents into searchable, AI-ready vector embeddings stored in Neon PostgreSQL. This is the foundation that gives KnowOps AI its *knowledge*: without it, the system cannot answer any operational questions. Every future RAG search query, grounded AI answer, and agent tool call depends directly on the data loaded in this phase.

> **Analogy:** If Phase 2.1 built the *AI brain*, Phase 2.2 is the *textbook* that fills it. The ingestion pipeline reads your IT runbooks, splits them into bite-sized pieces, converts each piece into a mathematical vector that captures its meaning, and stores it in the database — permanently ready for semantic search.

2. Pipeline Architecture — How the Ingestion Works

The ingestion pipeline in scripts/ingest.py follows a strict, deterministic 6-step process for every document:

Step 1 — File Discovery

The pipeline scans data/docs/ for all *.md files. It can also be pointed at a single file via --file or wiped and re-run from scratch with --reset.

Step 2 — Metadata Extraction

Each document's **title** is extracted from its top-level # Heading. A relative source path acts as a stable, unique document identifier for upsert logic.

Step 3 — Heading-Based Chunking

The document is split on ## and ### Markdown headings. This keeps each chunk semantically coherent — a chunk about 'Database Failover Steps' stays together as a single unit rather than being cut mid-sentence.

Step 4 — Overlap Sub-Chunking

Sections longer than 1,200 characters are further split with a 150-character overlap window. The overlap prevents loss of context at chunk boundaries — e.g., a procedure that spans two chunks still shares its intro sentence in both.

Step 5 — Embedding via Gemini

embed_document(chunk_text) from Phase 2.1 is called for each chunk. The Gemini text-embedding-004 model returns a 768-dimensional float vector that mathematically encodes the semantic meaning of that chunk.

Step 6 — Database Upsert

The document row is inserted (or updated if already exists) in the documents table. All old chunks are deleted and fresh chunks with their embeddings, headings, and ordering index are inserted into chunks.

3. Chunking Strategy — Design Decisions Explained

Strategy	Value / Setting	Why This Was Chosen
Primary split: Markdown headings	## / ### boundaries	Runbook sections are already semantically grouped by authors. Respecting these boundaries keeps related instructions together.
Secondary split: Character-level with overlap	1,200 chars / 150-char overlap	Prevents embeddings from exceeding the model's token limit while preserving cross-boundary context so no step in a procedure is orphaned.
Minimum chunk size	80 characters	Filters out stub chunks (e.g., a heading with no body) that would produce low-quality embeddings and waste API quota.
Embedding model	text-embedding-004, 768d	768 dimensions fit within pgvector's HNSW index limit of 2,000. Higher dimensions increase accuracy but inflate storage and index time.
Rate-limit pause	0.5s between API calls	Prevents hitting Gemini free-tier rate limits (60 QPM) during large batch ingestion runs.

4. Knowledge Base Documents Loaded

Four production-grade IT operations runbooks were created and ingested into the Neon database. These represent real categories of operational knowledge that KnowOps AI will answer questions about:

Document	File	Chunks	Topics Covered
Disaster Recovery Runbook	disaster_recovery_runbook.md	12	RTO/RPO targets, incident classification (P1–P4), first-response steps, PostgreSQL manual failover, DNS cutover, Kubernetes rollback, PIR process, escalation matrix.
On-Call Guide & Incident Response	on_call_guide.md	17	On-call responsibilities, PagerDuty/Grafana/Datadog tooling, alert triage decision tree, 5 common alert runbooks (CPU, DB pool, SSL, latency, Gemini 503), shift handoff checklist.
Kubernetes Operations Guide	kubernetes_operations.md	16	EKS cluster management, deployment / rollback / canary strategies, HPA auto-scaling config, node drain/uncordon, CrashLoopBackOff debugging, secrets management via AWS CSI Driver, resource quotas.
API Troubleshooting Guide	api_troubleshooting.md	16	HTTP error codes (400–503), JWT auth issues, /ask endpoint failure modes, slow vector search diagnosis, memory usage, rate limiting (429), structured log format, DB query debugging SQL.
TOTAL	4 documents	**61**	Complete IT operations knowledge base, 0 failures.

5. Database Schema Changes in Phase 2.2

Two new columns were added to the chunks table in Neon PostgreSQL to support ordering and section-aware retrieval:

```
ALTER TABLE chunks ADD COLUMN chunk_index INTEGER NOT NULL DEFAULT 0;
ALTER TABLE chunks ADD COLUMN heading TEXT NOT NULL DEFAULT '';
```

Column	Type	Purpose
chunk_index	INTEGER	Position of the chunk within its document (0, 1, 2...). Used to reconstruct reading order and display adjacent context during retrieval.
heading	TEXT	The Markdown section heading the chunk belongs to (e.g., '4.1 Detecting Primary Database Failure'). Shown in API responses as source citations so users know exactly which section was used.

6. Live Verification Results

After ingestion, a live semantic search was run against the Neon database to confirm the vector index is functional and retrieving the correct content:

Test Query: *"How do I perform a database failover?"*

Rank	Similarity Score	Document	Section / Heading
#1	**0.7147**	Disaster Recovery Runbook	4.3 Verification — post-failover confirmation steps
#2	0.6967	Disaster Recovery Runbook	4.1 Detecting Primary Database Failure — alert signs and indicators
#3	0.6804	Disaster Recovery Runbook	Manual failover steps — DNS CNAME cutover with aws route53

■ **Interpretation:** All top-3 results correctly identify the Disaster Recovery Runbook as the most relevant document — and specifically the failover-related sections. The similarity scores (0.68–0.71) are strong, confirming the embedding model has correctly encoded the semantic meaning of both the query and the stored chunks. An unrelated query like 'baking a cake' would score < 0.2 against these chunks.

7. How Phase 2.2 Feeds the Rest of the Platform

The data loaded in Phase 2.2 is consumed by every subsequent phase:

Phase / Feature	Consumes 2.2 Data Via	What It Does With It
Phase 2.3 — RAG Retrieval app/rag.py	SQL: ORDER BY embedding <=> query_vec LIMIT 5	Finds the 5 most semantically similar chunks to a user query using HNSW cosine search. Returns chunk text + heading + source as grounded context.

Phase 2.4 — RAG Evaluation tests/eval_rag.py	Retrieves chunks, compares vs. ground truth	Tests retrieval precision. Confirms the right document section appears in top-k results for benchmark queries.
Phase 2.5 — /ask API Endpoint app/main.py	RAG retrieval → chat_completion(context_chunks)	End-to-end: user submits a question → system retrieves relevant chunks from the KB → Gemini generates a grounded answer citing the correct runbook section.
Phase 3 — AI Agent app/agent.py	Knowledge base as a tool in the agent's tool belt	The agent calls the RAG retrieval tool autonomously during multi-step reasoning — e.g., fetching the on-call guide mid-conversation to answer a complex incident question.

8. Running the Ingestion Pipeline — CLI Reference

Ingest all docs (first time):

```
python scripts/ingest.py
```

Re-ingest a single updated file:

```
python scripts/ingest.py --file data/docs/on_call_guide.md
```

Wipe + full re-ingest (after schema change):

```
python scripts/ingest.py --reset
```

Quiet mode (no per-chunk output):

```
python scripts/ingest.py --quiet
```

9. Next Steps: Phase 2.3 — RAG Retrieval

With 61 chunks stored and indexed in Neon, Phase 2.3 will build app/rag.py — the retrieval layer that takes a user's natural language question, converts it to a query embedding using embed_query(), runs a cosine similarity search against the HNSW index, and returns the top-k most relevant chunks with their source metadata.

Phase 2.3 will implement:

- retrieve(query: str, top_k: int = 5) → list[ChunkResult] — the core RAG function
- Threshold filtering — only return chunks with similarity > 0.5 to avoid noise
- Source citation metadata — document title + section heading per result
- Integration into chat_completion() as the context_chunks input